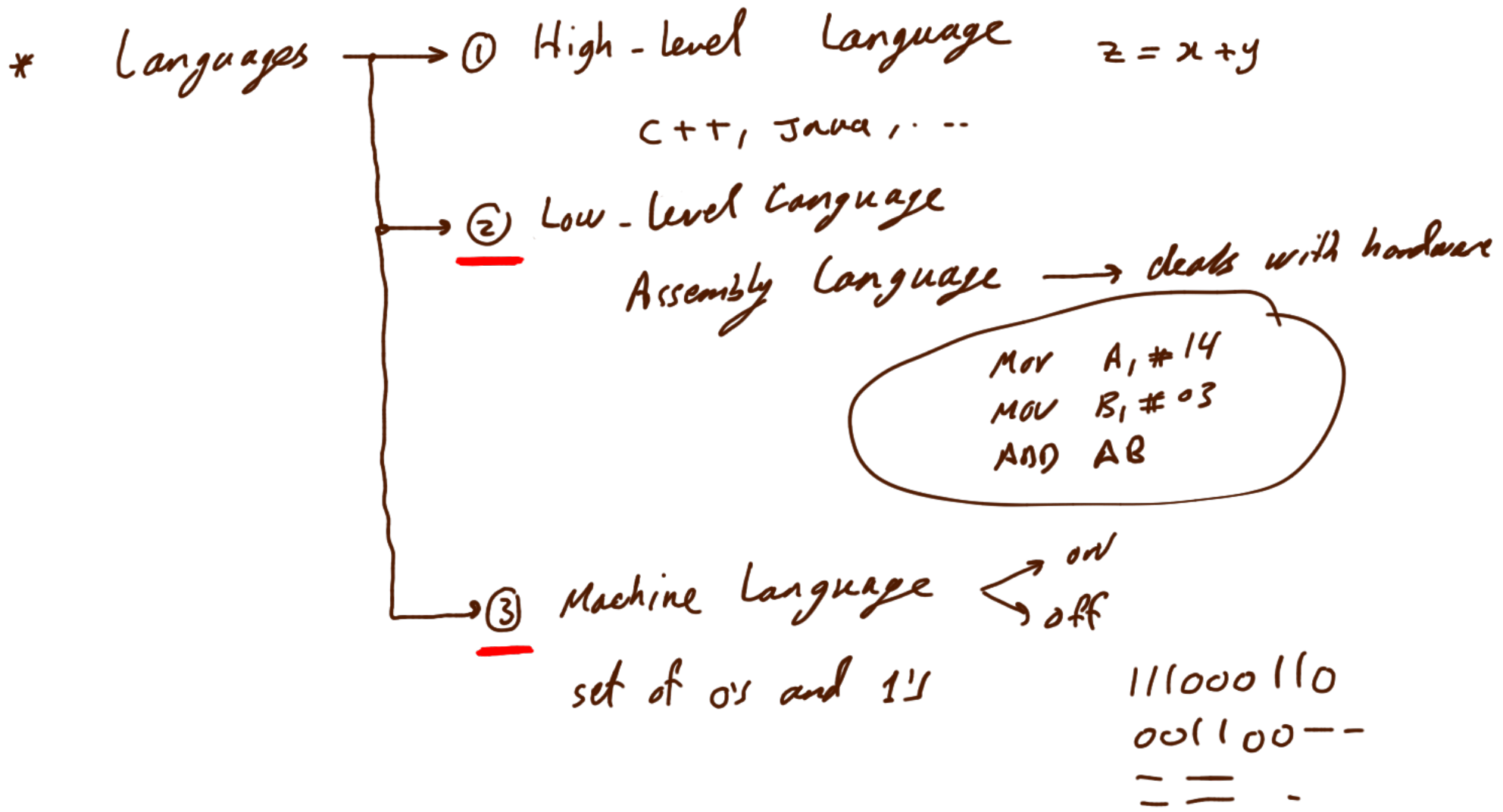




-
- The CPU of 8051 works only on binary values (0's and 1's)
 - program consists of 0's and 1's is called machine language program.
 - Any Assembly program must be converted (translated) into machine language program. using program called Assembler.

Assembly

MOV A, R2
ADD A, R3
⋮

Assembler
program

machine language

110010011
1000010101
⋮

(2)

- Any assembly line can be as follows

[Label:] mnemonic [operands] [; comments] //

label name *keyword* *optional* *or*

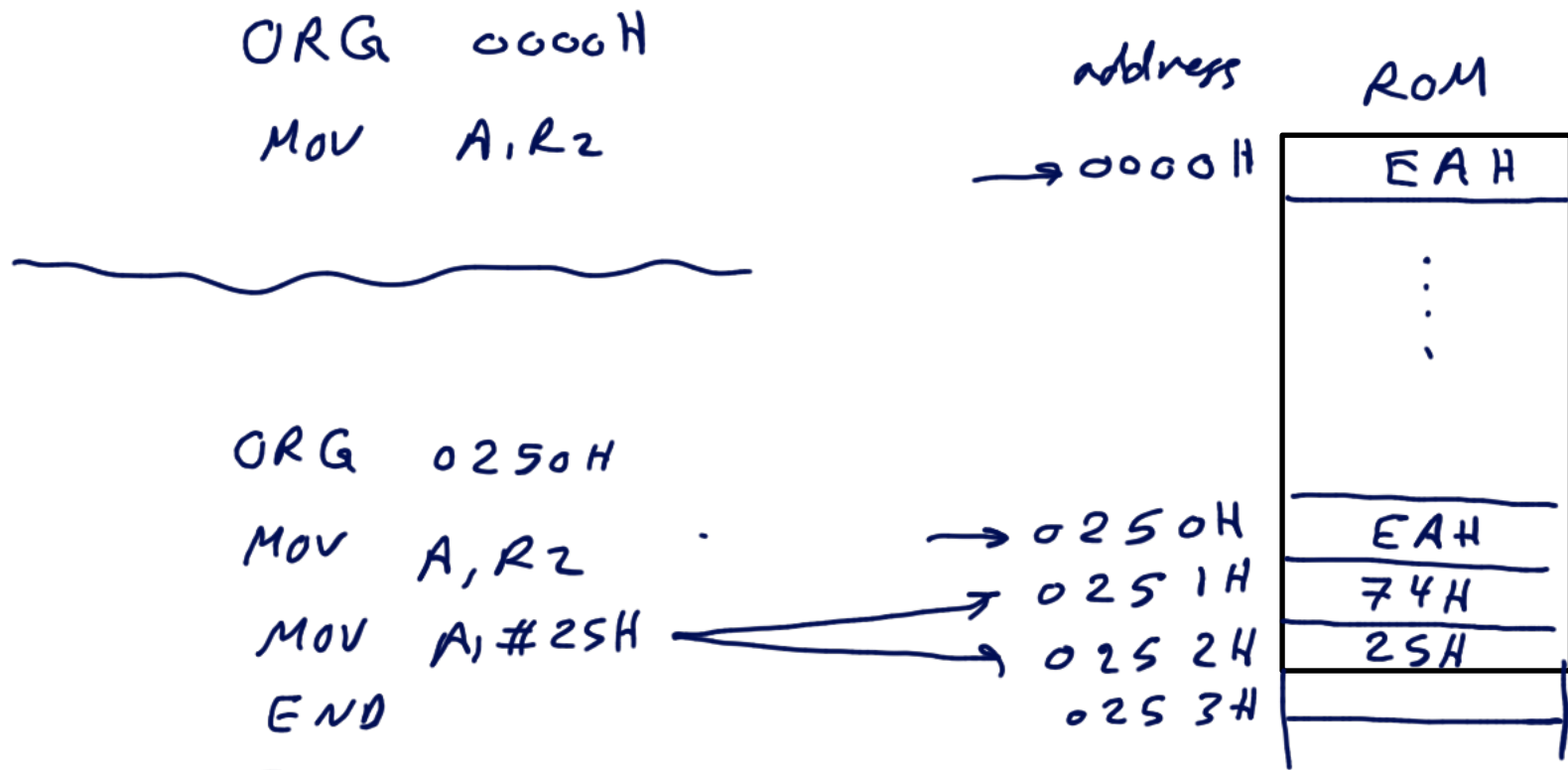
NEXT: MOV R2, A ; copy A in R2

- Any assembly line can be
 - instruction
 - directive

- Directives (pseudo instructions): Do not generate machine languages (code) but they are used by assembler itself.

- Comments don't generate any machine code.

① ORG : (origin) ^{the assembler}
 is used to indicate ^{the} beginning address of source file.

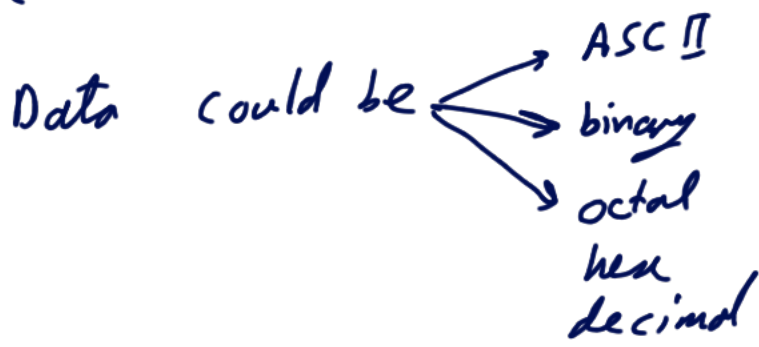


② END : is used to indicate to the assembler the end of source file.

	address	ROM		
1	0000	↓	ORG 0H	;start at location 0
2	0000	7D25	MOV R5, #25H	;load 25H into R5
3	0002	7F34	MOV R7, #34H	;load 34H into R7
4	0004	7400	MOV A, #0	;load 0 into A
5	0006	2D	ADD A, R5	;add contents of R5 to A
				;now A = A + R5
6	0007	2F	ADD A, R7	;add contents of R7 to A
				;now A = A + R7
7	0008	2412	ADD A, #12H	;add to A value 12H
				;now A = A + 12H
8	000A	80FE	HERE: SJMP HERE	;stay in this loop
9	000C		END	;end of asm source file

③ DB : Define byte

is used to define 8-bit (1 byte) of data in memory (ROM).



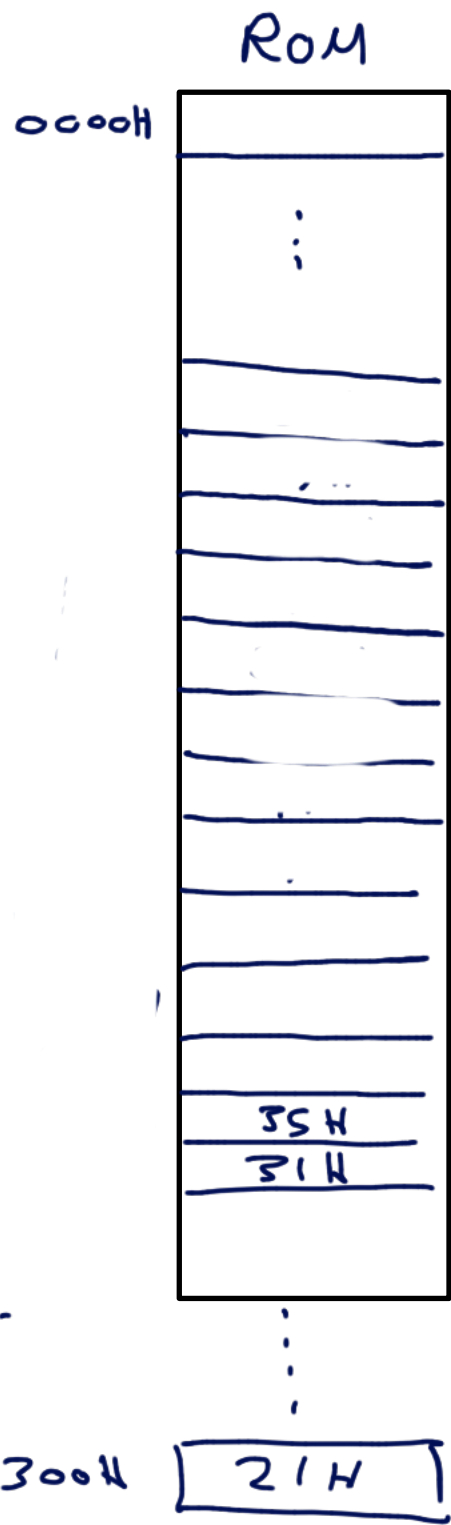
```
ORG 197H
DB 28H
DB 25
DB 1001100B
DB 'F'
DB '2'

DB 0E4H
```

```
DB "Hi 8051"
```

```
ORG 300H
DB 21H
```

```
ORG 197H
DB 28H, 25, 1001100B, 'F', .....
```



- ④ EQU : equate $\xrightarrow{\text{byte or more}}$ is used to define constant without occupying a memory location in ROM. ⑤

NUM EQU 15H
:

MOV R2, #NUM

ADD A, #NUM

- ⑤ Bit : is similar to EQU but it is used with one bit

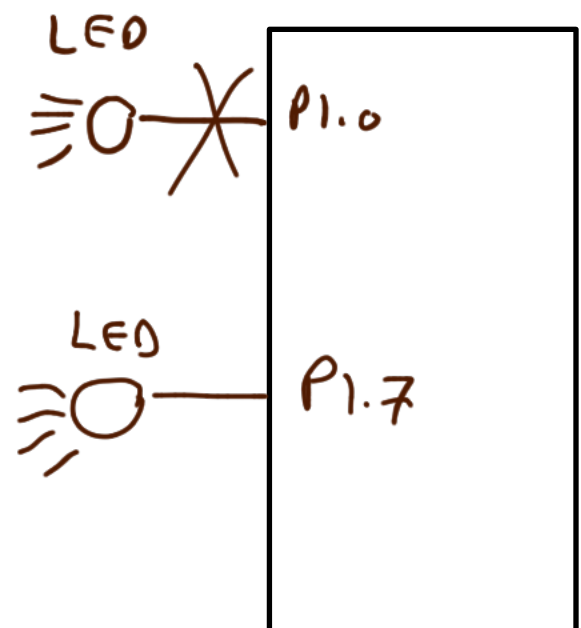
SETB P1.0

CLR P1.0

LED Bit P1.0

SETB LED

CLR LED



Stack is used to store
data and addresses temporarily

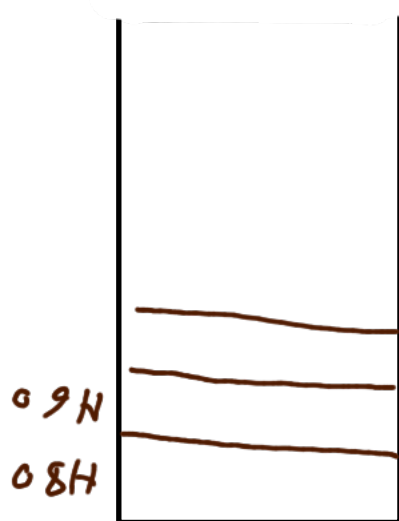
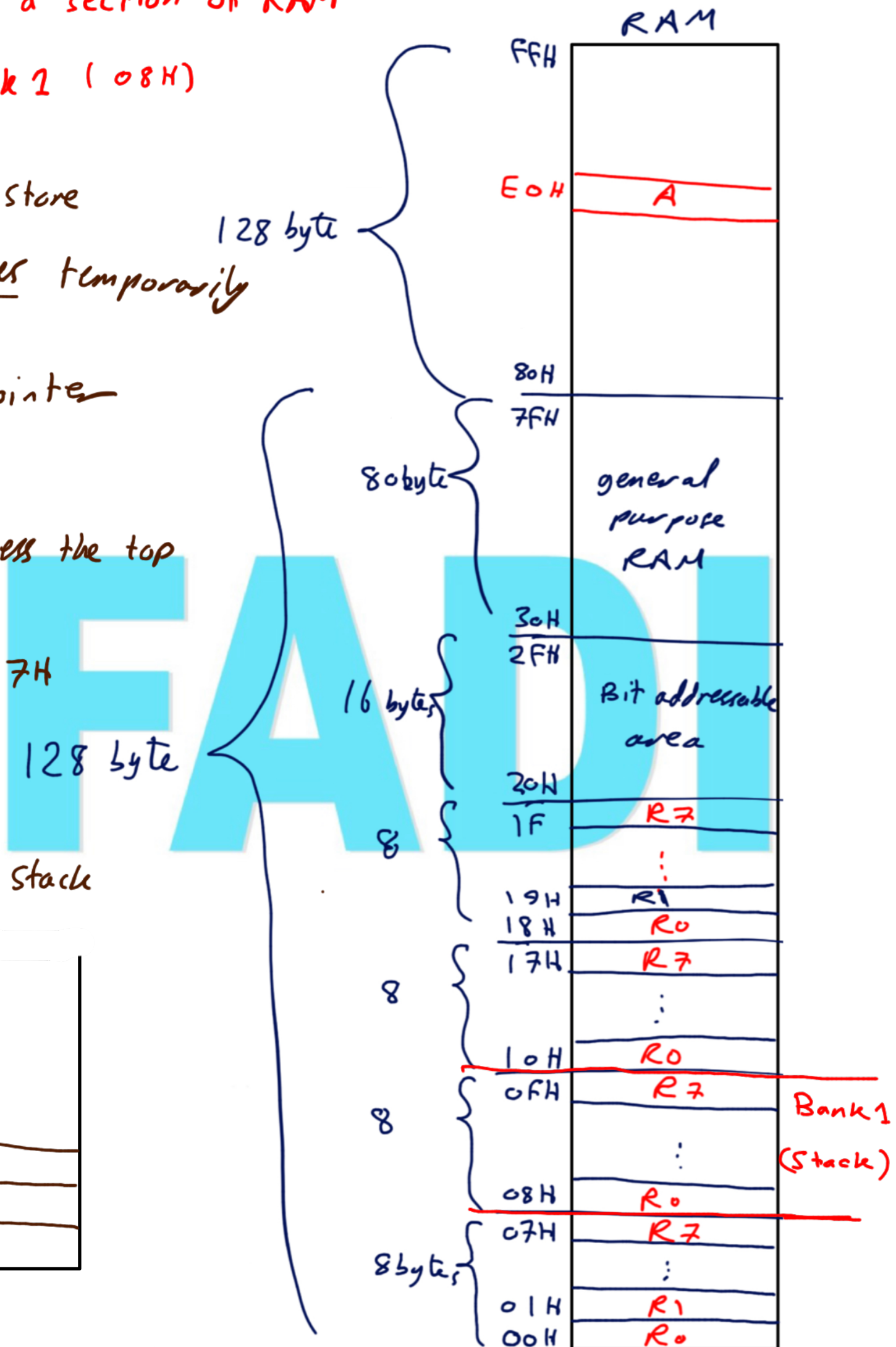
* SP : stack pointer

(8-bit)

is used to access the top of the stack

by default $sp = 0.7H$

empty stack


$$Sp \rightarrow O(7,1)$$


Stack is LIFO : Last-in - First-out

⑧

① PUSH : is used to store data in stack from Reg. or memory location.

$$\textcircled{1} \quad SP = SP + \underline{\underline{1}}$$

$$\textcircled{2} \quad [SP] = \text{mem}$$

increment SP
by 1

(pull)
② POP : is used to retrieve the top of stack into memory location

$$\textcircled{1} \quad \text{mem} = [SP]$$

$$\textcircled{2} \quad SP = SP - \underline{\underline{1}}$$

decrement SP
by 1

show me stack and stack pointer (SP) after executing (9)
the following code.

MOV R2, # 50H

MOV R3, # 60H

MOV R7, # 'E'

PUSH 2

PUSH 3

PUSH 7

MOV R7, # 14H

POP 7

POP 3

POP 2

SP → 07H

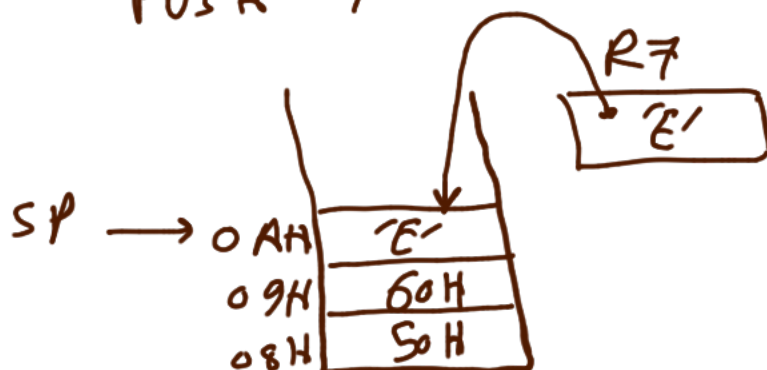
after push 2



after PUSH 3

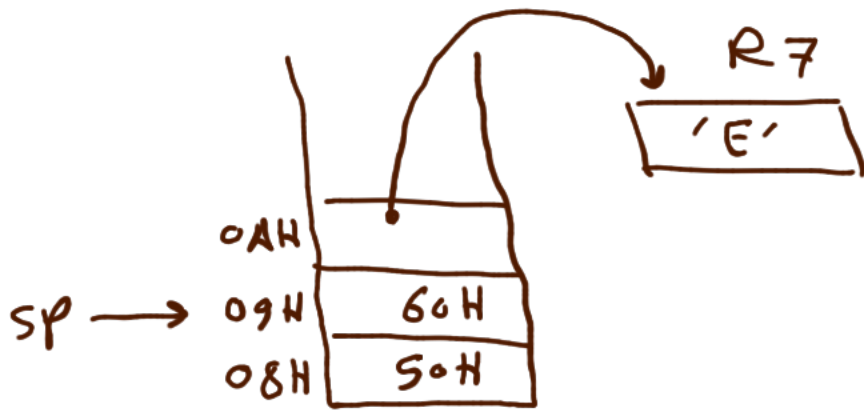


PUSH 7

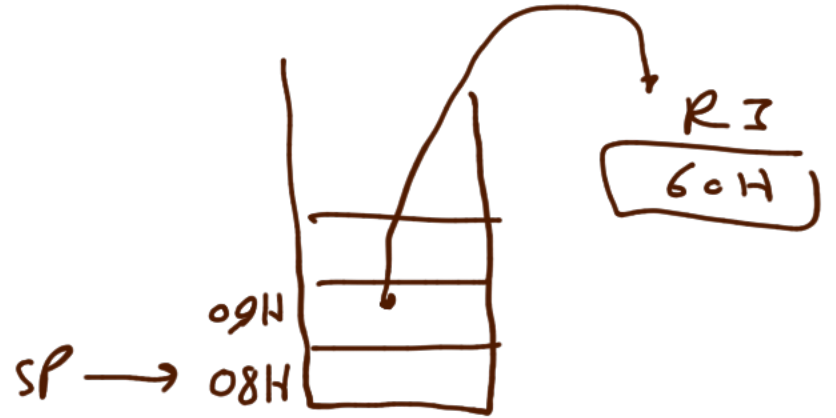


R7
14H

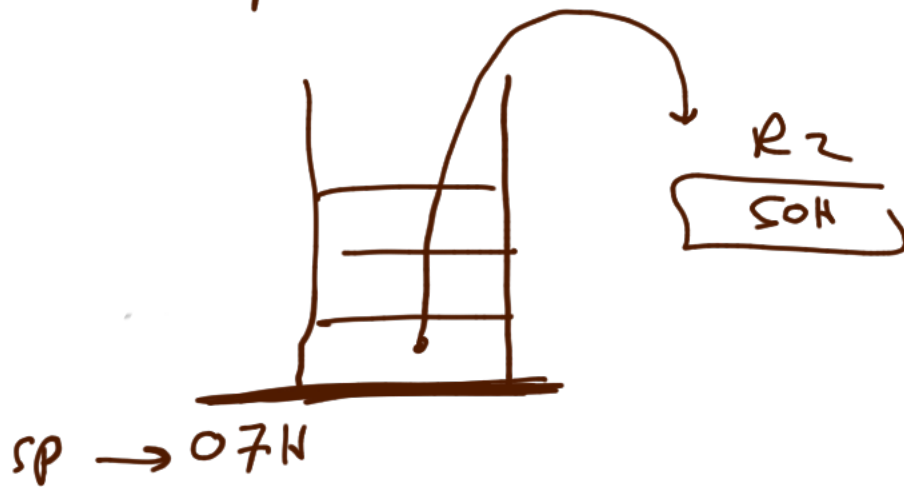
after pop 7



pop 3



pop 2



Stack locations in RAM

① start at bank 1 (08H → 1FH) → bank 3

② scratch pad RAM (30H → 7FH)

using instruction

MOV SP, #2FH

- Bit addressable area (20H → 2FH) is not allowed for stack.

MOV R4, #28H ←

MOV SP, #59H ←

PUSH 4

